RAPPORT DE TRAVAUX PRATIQUES

UNIVERSITÉ SULTAN MOULAY SLIMANE
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES
BÉNI MELLAL

TP4 – Architecture Orientée Services

Réalisé par : EL MELIANI Ammar
Encadrant : Pr. EL BAGHAZAOUI

Filière : **IACS-S2**
Année universitaire : 2025/2026

Lien GITHUB :

https://github.com/EL-Star456/TP_Rapport_Log.git

TP 4 – Architecture Orientée Services (SAO)

1. Introduction :

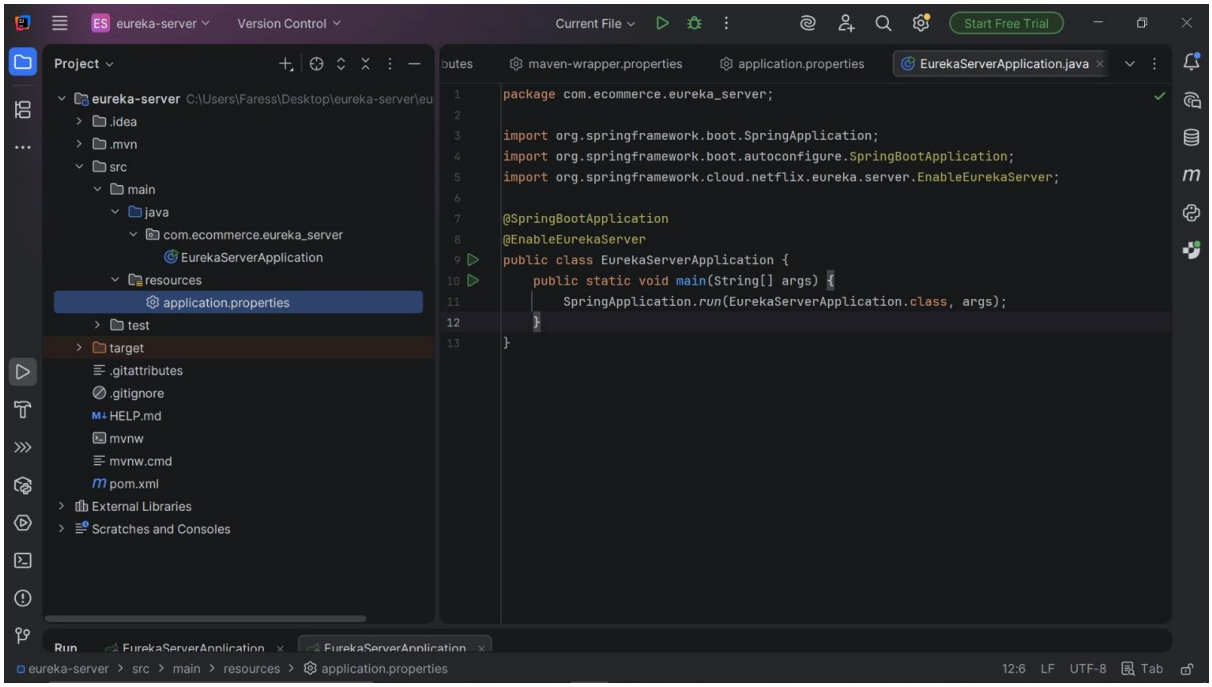
L'architecture orientee services (SOA - Service-Oriented Architecture) est un paradigme de conception logicielle qui permet de structurer une application en services independants et reutilisables. Dans ce TP, nous avons cree une application e-commerce utilisant Spring Boot pour implementer une architecture SOA.

SOA represente une evolution naturelle par rapport au monolithe (TP1/TP2) : au lieu d'avoir une seule application qui fait tout, on decoupe le systeme en services independants qui communiquent entre eux via des APIs REST.

2. Objectifs Pédagogiques :

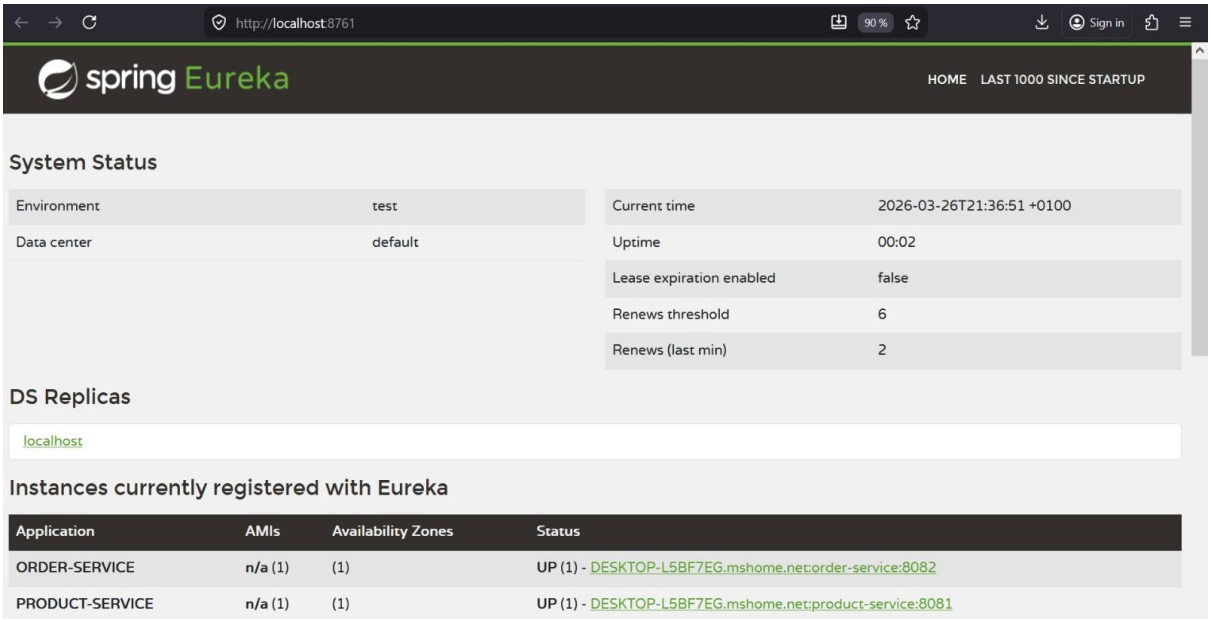
- Comprendre les principes de l'architecture SOA
- Creer des microservices avec Spring Boot
- Implementer la communication inter-services via REST (OpenFeign)
- Utiliser un service de decouverte Eureka pour l'enregistrement des services
- Gerer une base de donnees avec Spring Data JPA (H2 en memoire)

3. Etape 1 - Eureka Server :

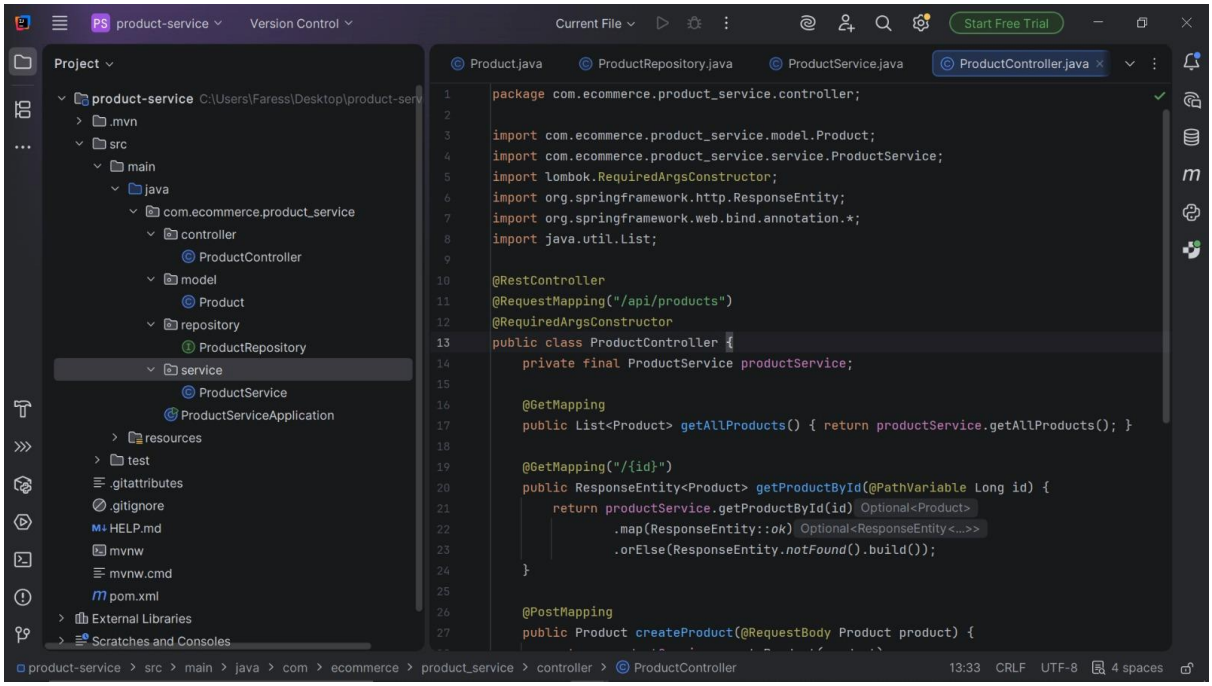


3.1 Tableau de bord Eureka :

Apres démarrage, le tableau de bord Eureka est accessible sur http://localhost:8761. Il affiche tous les services enregistres avec leur statut (UP/DOWN), leur adresse et leur port. On peut voir PRODUCT-SERVICE et ORDER-SERVICE enregistres avec le statut UP.



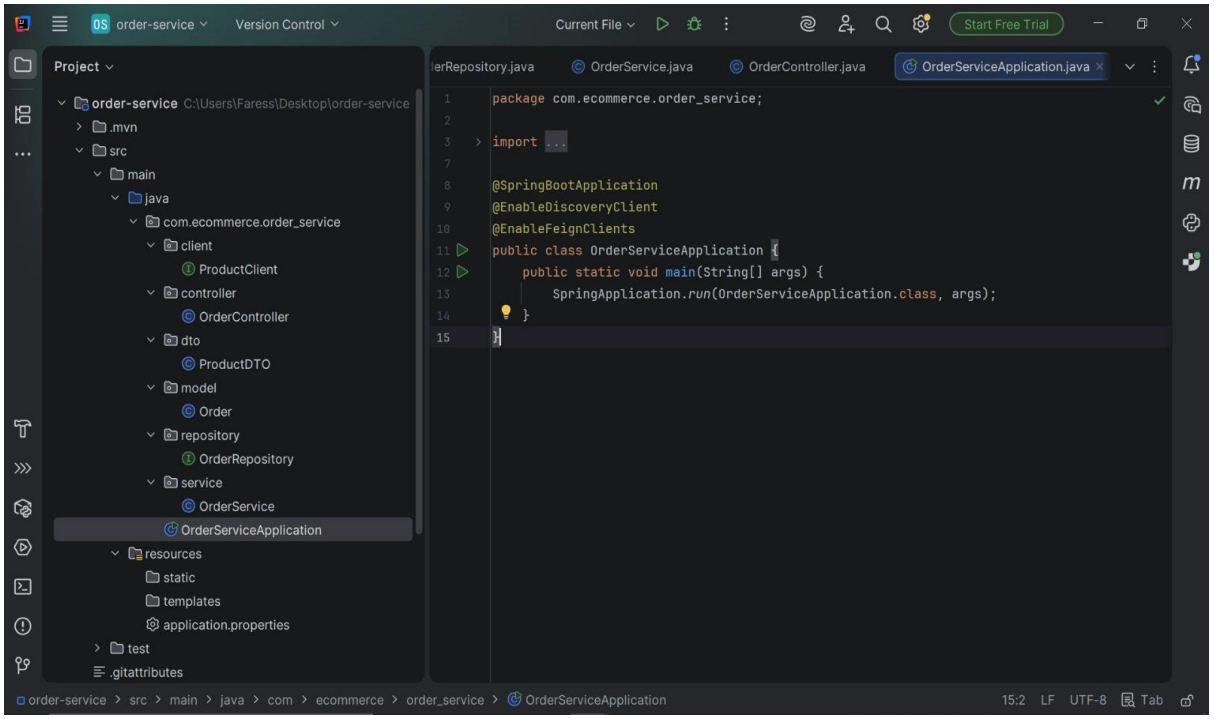
4. Etape 2 - Product Service :



5. Etape 3 - Order Service :

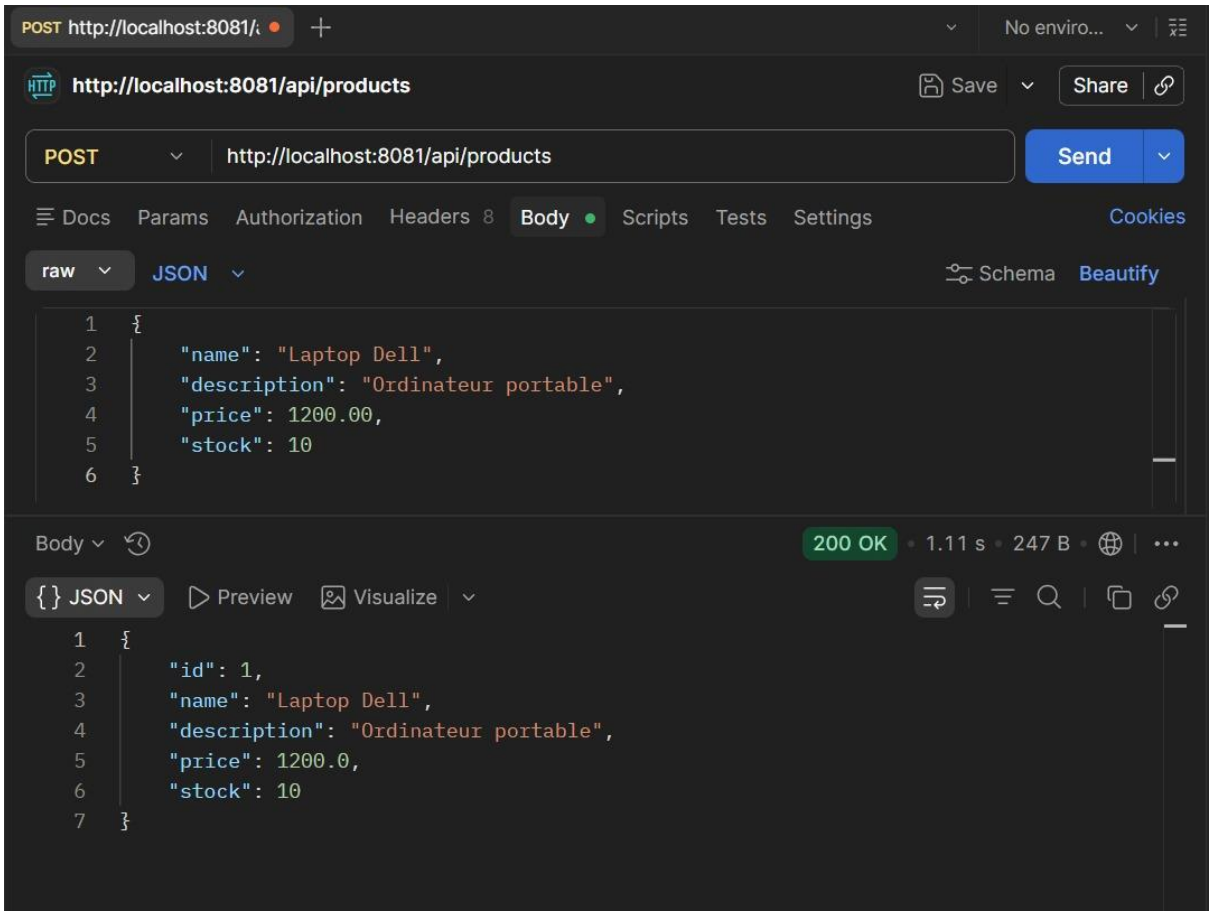
5.1 OpenFeign :

OpenFeign est un client HTTP déclaratif de Spring Cloud. Il permet d'appeler un autre microservice comme si c'était une méthode Java locale, sans écrire de code HTTP manuellement. OpenFeign utilise Eureka pour trouver l'adresse de Product Service automatiquement.



6. Tests avec Postman :

6.1 Test 1 :



6.2 Test 2:j

GET http://localhost:8081/a | GET http://localhost:8081/a | No enviro...

http://localhost:8081/api/products | Save | Share

GET | http://localhost:8081/api/products | Send

Docs | Params | Authorization | Headers 6 | Body | Scripts | Tests | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | 200 OK | 17 ms | 249 B

{} JSON | Preview | Visualize


```
[
    {
        "id": 1,
        "name": "Laptop Dell",
        "description": "Ordinateur portable",
        "price": 1200.0,
        "stock": 10
    }
]
```


http://localhost:8082/api/orders | Save | Share

POST | http://localhost:8082/api/orders?productId=1&quantity=2 | Send

Docs | Params | Authorization | Headers 7 | Body | Scripts | Tests | Settings | Cookies

Query Params

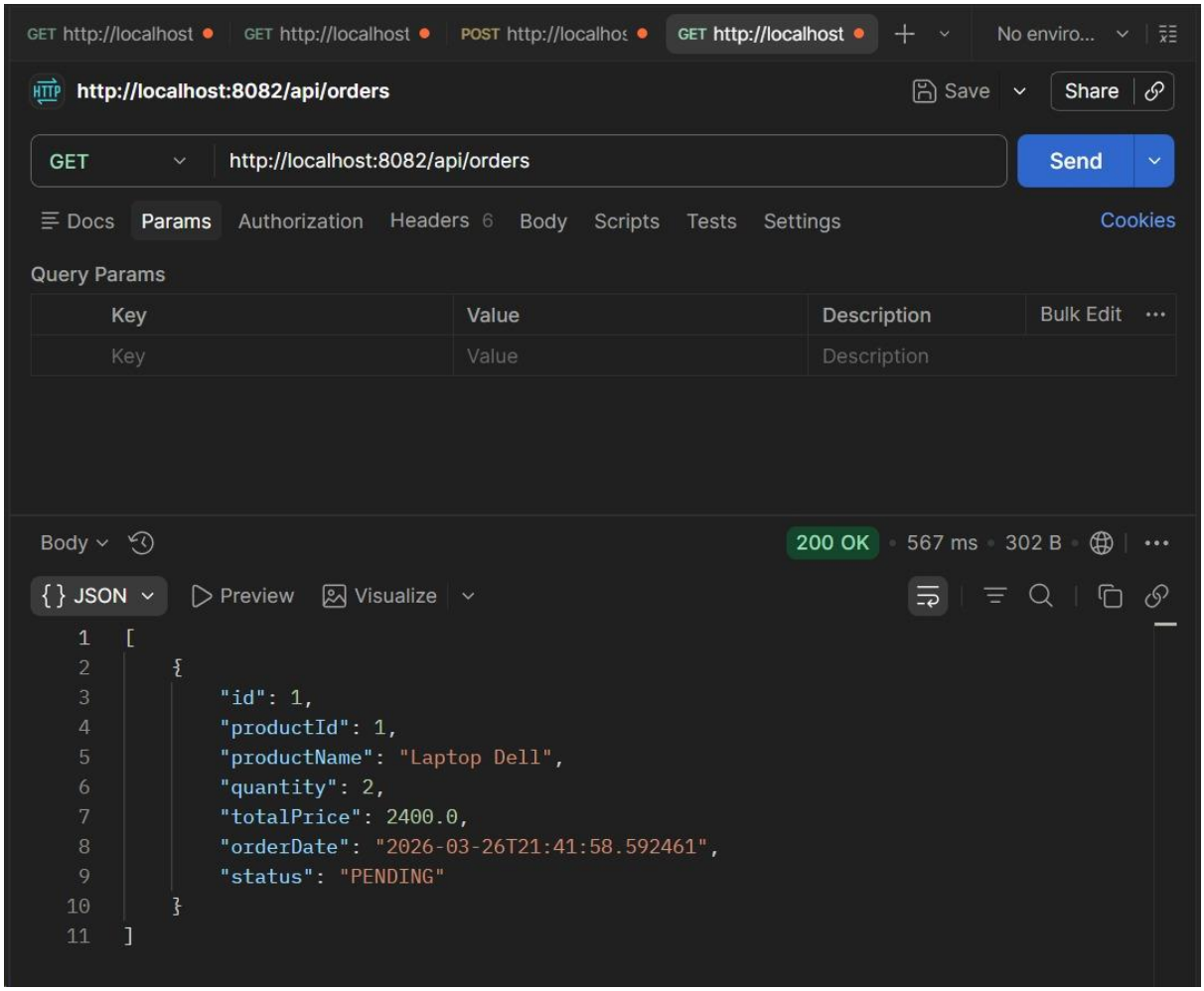
Key	Value	Description	Bulk Edit
✓ productId	1		
✓ quantity	2		
Key	Value	Description	

Body | 200 OK | 1.54 s | 301 B

{} JSON | Preview | Visualize


```
{
    "id": 1,
    "productId": 1,
    "productName": "Laptop Dell",
    "quantity": 2,
    "totalPrice": 2400.0,
    "orderDate": "2026-03-26T21:41:58.5924606",
    "status": "PENDING"
}
```

6.4 Test 4:



11. Conclusion:

Le TP4 a introduit l'architecture SOA avec Spring Boot, Eureka et OpenFeign. Cette architecture represente une evolution majeure par rapport au monolithe des TPs precedents.

Les avantages principaux sont l'indépendance des services (chaque service peut etre déployé, mis a jour ou scalie indépendamment), l'isolation des pannes (si Order Service tombe, Product Service continue de fonctionner), et la scalabilité sélective (on peut lancer plusieurs instances de Product Service si nécessaire).

Fin